Task 4

# Compare Model Accuracy and Performance Metrics

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

**Objective.** Compare all trained models with consistent metrics on unseen data and select the best one.

## 1. Method

All 7 saved models were scored on the **same held-out test set** (6,000 customers, 26.9% churn) that was never used for fitting or tuning. Eight metrics were computed. Because only 73.1% of customers stay if we predict 'no churn' for everybody, accuracy alone is misleading (baseline = 0.731); ROC-AUC, PR-AUC and recall on the churn class matter more.

## 2. Results (threshold 0.5, sorted by ROC-AUC)

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC | PR-AUC | Log-loss | Brier |
|---|---|---|---|---|---|---|---|---|
| Logistic Regression | 0.774 | 0.636 | 0.379 | 0.475 | 0.791 | 0.577 | 0.471 | 0.155 |
| Gradient Boosting (tuned) | 0.769 | 0.624 | 0.358 | 0.455 | 0.787 | 0.570 | 0.474 | 0.156 |
| Neural Network (MLP) | 0.767 | 0.617 | 0.353 | 0.449 | 0.786 | 0.568 | 0.475 | 0.157 |
| Gradient Boosting | 0.766 | 0.614 | 0.354 | 0.449 | 0.782 | 0.563 | 0.478 | 0.158 |
| Random Forest | 0.767 | 0.627 | 0.332 | 0.434 | 0.780 | 0.559 | 0.480 | 0.158 |
| k-NN | 0.765 | 0.629 | 0.307 | 0.413 | 0.762 | 0.539 | 0.526 | 0.163 |
| Decision Tree | 0.756 | 0.576 | 0.363 | 0.445 | 0.751 | 0.505 | 0.517 | 0.167 |

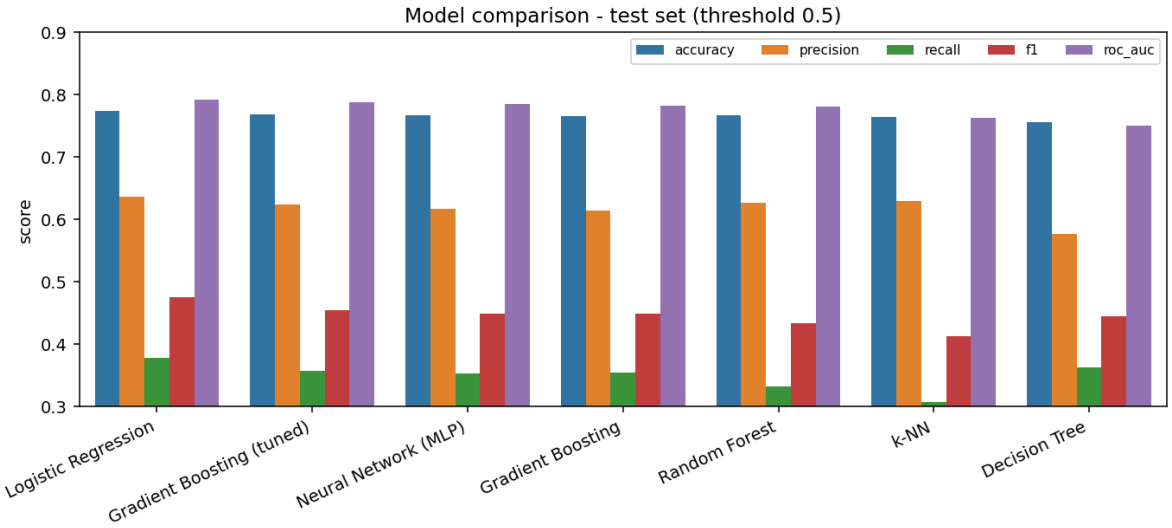



Figure 1 - Metrics for all models

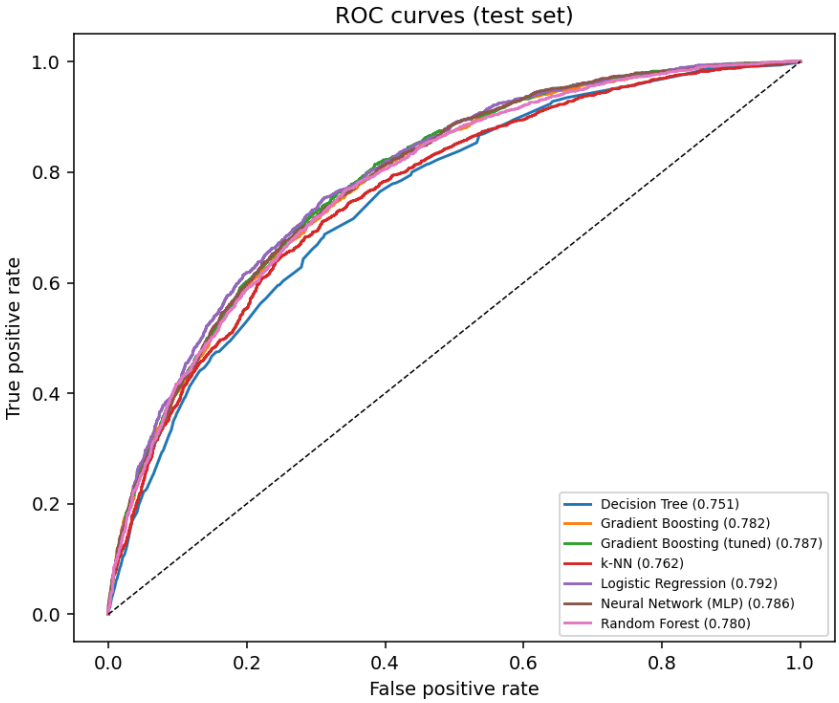



Figure 2 - ROC curves

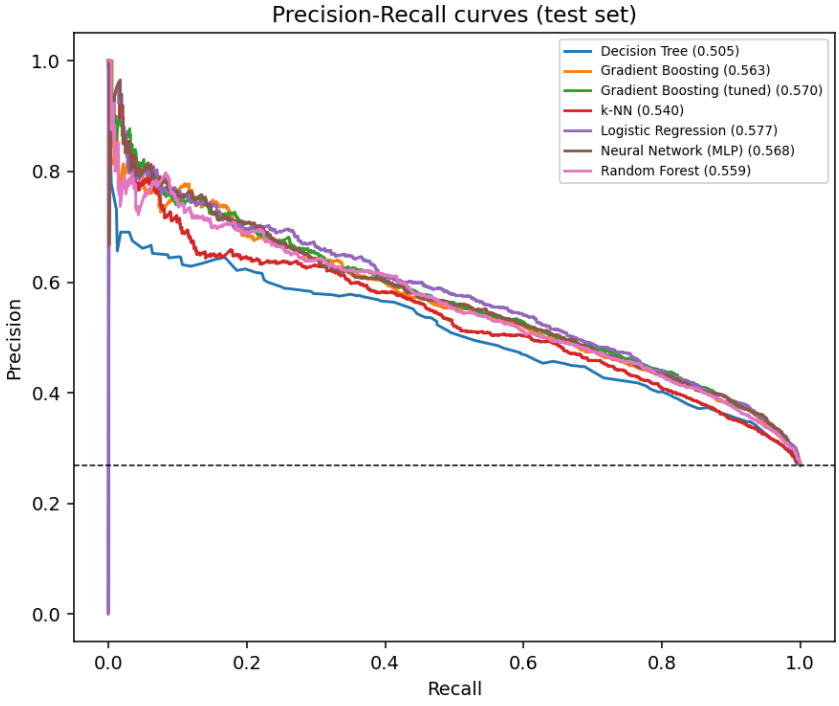



Figure 3 - Precision-Recall curves

## 3. Model selection and decision threshold

**Selected model: Logistic Regression** (highest ROC-AUC 0.791, lowest log-loss and Brier score, i.e. also the best-calibrated probabilities). At the default 0.5 threshold recall for churners is low (38%) because churners are a minority. The threshold was therefore tuned to maximise F1 on *out-of-fold training predictions* (not on the test set): **threshold = 0.30**.

| Metric at threshold 0.30 | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Test set | 0.727 | 0.495 | 0.684 | 0.574 | 0.791 |

Confusion matrix on the test set: 3256 true negatives, 1128 false positives, 511 false negatives, 1105 true positives. The lower threshold trades some precision for far more churners caught (recall 68%), which is the right trade when a retention offer is cheap compared with losing a customer.

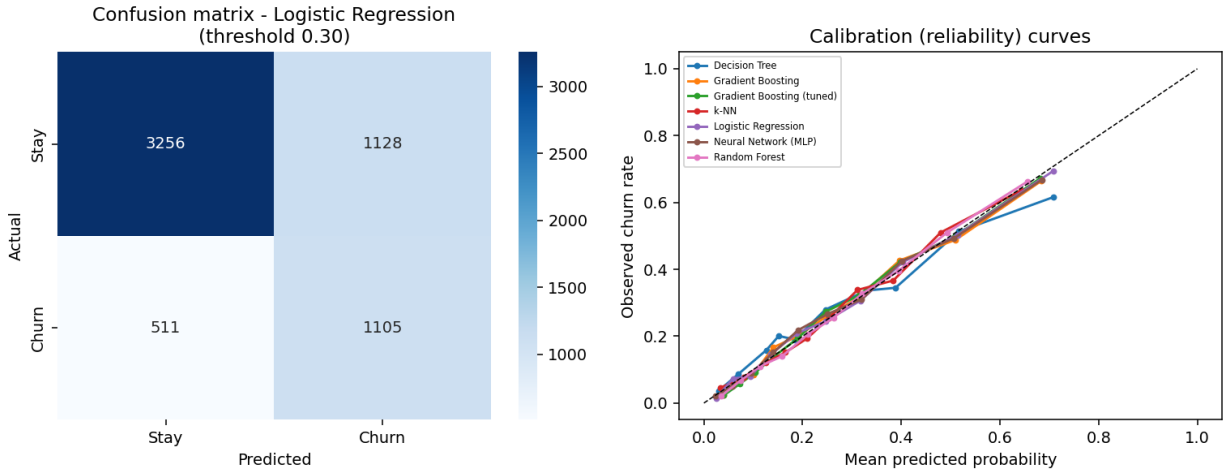



Figure 4 - Confusion matrix (selected model) and calibration curves

## 4. Interpretation and caveats

- Differences between the top models are small (ROC-AUC 0.78-0.79). Simple logistic regression wins because the synthetic churn mechanism is close to additive; on real, messier data non-linear models (boosting) often win - the pipeline makes it a one-line change.
- Test-set size (6,000) gives an AUC standard error of roughly 0.007, so the top three models are statistically close; logistic regression is preferred also for interpretability and speed.
- A ceiling near 0.79 AUC is expected: the generator adds random noise to the churn probability, so some churn is inherently unpredictable.

# Appendix - Source code

## src/compare_models.py

```
"""TASK 4 - Compare model accuracy and performance metrics on the held-out test set.

All saved pipelines are scored on data never used for fitting/tuning. The best model
(by ROC-AUC) is promoted to models/best_model.joblib; its decision threshold is chosen
on out-of-fold TRAIN predictions (maximising F1) to avoid tuning on the test set.
"""
import json
from datetime import datetime, timezone
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.calibration import calibration_curve
from sklearn.metrics import (accuracy_score, average_precision_score, brier_score_loss, confusion_matrix,
                             f1_score, log_loss, precision_recall_curve, precision_score, recall_score,
                             roc_auc_score, roc_curve)
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from .config import DATA_PROC, MODELS, RESULTS, FIGS, SEED, TARGET
from .features import CATEGORICAL, NUMERIC

PRETTY = {"logistic_regression": "Logistic Regression", "decision_tree": "Decision Tree",
          "random_forest": "Random Forest", "gradient_boosting": "Gradient Boosting",
          "gradient_boosting_tuned": "Gradient Boosting (tuned)", "knn": "k-NN",
          "mlp_neural_net": "Neural Network (MLP)"}


def metrics(y, p, thr=0.5):
    pred = (p >= thr).astype(int)
    return {"accuracy": accuracy_score(y, pred), "precision": precision_score(y, pred),
            "recall": recall_score(y, pred), "f1": f1_score(y, pred), "roc_auc": roc_auc_score(y, p),
            "pr_auc": average_precision_score(y, p), "log_loss": log_loss(y, p),
            "brier": brier_score_loss(y, p)}


def main():
    test = pd.read_csv(DATA_PROC / "test.csv"); train = pd.read_csv(DATA_PROC / "train.csv")
    Xte, yte = test.drop(columns=[TARGET]), test[TARGET].values
    probs, rows = {}, []
    for f in sorted(MODELS.glob("*.joblib")):
        if f.stem == "best_model":
            continue
        pipe = joblib.load(f)
        p = pipe.predict_proba(Xte)[:, 1]
        probs[f.stem] = p
        rows.append({"model": f.stem, "name": PRETTY.get(f.stem, f.stem), **metrics(yte, p)})
    df = pd.DataFrame(rows).sort_values("roc_auc", ascending=False).reset_index(drop=True)
    df.round(4).to_csv(RESULTS / "model_comparison.csv", index=False)
    best = df.loc[0, "model"]

    best_pipe = joblib.load(MODELS / f"{best}.joblib")
    oof = cross_val_predict(best_pipe, train.drop(columns=[TARGET]), train[TARGET],
                            cv=StratifiedKFold(3, shuffle=True, random_state=SEED), method="predict_proba")[:, 1]
    grid = np.linspace(0.1, 0.7, 61)
    thr = float(grid[int(np.argmax([f1_score(train[TARGET], oof >= t) for t in grid]))])
    bm = metrics(yte, probs[best], thr)
    joblib.dump(best_pipe, MODELS / "best_model.joblib")
    meta = {"best_model": best, "best_model_name": PRETTY.get(best, best), "threshold": round(thr, 3),
            "test_metrics_at_threshold": {k: round(float(v), 4) for k, v in bm.items()},
            "trained_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
            "numeric_features": NUMERIC, "categorical_features": CATEGORICAL, "target": TARGET}
    (MODELS / "metadata.json").write_text(json.dumps(meta, indent=2))
    cm = confusion_matrix(yte, probs[best] >= thr)

    # ---------- figures ----------
    pal = sns.color_palette("tab10", len(df))
    plt.figure(figsize=(6.5, 5.5))
    for c, (m, p) in zip(pal, probs.items()):
        fpr, tpr, _ = roc_curve(yte, p)
        plt.plot(fpr, tpr, color=c, label=f"{PRETTY.get(m, m)} ({roc_auc_score(yte, p):.3f})")
    plt.plot([0, 1], [0, 1], "k--", lw=.8); plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
    plt.title("ROC curves (test set)"); plt.legend(fontsize=7, loc="lower right"); plt.tight_layout()
    plt.savefig(FIGS / "t4_roc.png", dpi=140); plt.close()

    plt.figure(figsize=(6.5, 5.5))
    for c, (m, p) in zip(pal, probs.items()):
        pr, rc, _ = precision_recall_curve(yte, p)
        plt.plot(rc, pr, color=c, label=f"{PRETTY.get(m, m)} ({average_precision_score(yte, p):.3f})")
    plt.axhline(yte.mean(), color="k", ls="--", lw=.8); plt.xlabel("Recall"); plt.ylabel("Precision")
    plt.title("Precision-Recall curves (test set)"); plt.legend(fontsize=7); plt.tight_layout()
    plt.savefig(FIGS / "t4_pr.png", dpi=140); plt.close()

    long = df.melt(id_vars="name", value_vars=["accuracy", "precision", "recall", "f1", "roc_auc"])
    plt.figure(figsize=(10, 4.6))
    sns.barplot(data=long, x="name", y="value", hue="variable")
    plt.xticks(rotation=25, ha="right"); plt.ylim(0.3, 0.9); plt.xlabel(""); plt.ylabel("score")
```

```
    plt.title("Model comparison - test set (threshold 0.5)"); plt.legend(ncol=5, fontsize=8)
    plt.tight_layout(); plt.savefig(FIGS / "t4_metric_bars.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 2, figsize=(11, 4.4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax[0], xticklabels=["Stay", "Churn"],
                yticklabels=["Stay", "Churn"]); ax[0].set_title(f"Confusion matrix - {PRETTY.get(best, best)}\n(threshold
        {thr:.2f})")
    ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual")
    for c, (m, p) in zip(pal, probs.items()):
        fp, mp = calibration_curve(yte, p, n_bins=10, strategy="quantile")
        ax[1].plot(mp, fp, marker="o", ms=3, color=c, label=PRETTY.get(m, m))
    ax[1].plot([0, 1], [0, 1], "k--", lw=.8); ax[1].set_title("Calibration (reliability) curves")
    ax[1].set_xlabel("Mean predicted probability"); ax[1].set_ylabel("Observed churn rate"); ax[1].legend(fontsize=6)
    plt.tight_layout(); plt.savefig(FIGS / "t4_confusion_calibration.png", dpi=140); plt.close()

    out = {"table": df.round(4).to_dict("records"), "best_model": best, "threshold": thr,
           "best_at_threshold": meta["test_metrics_at_threshold"], "confusion_matrix": cm.tolist(),
           "test_rows": int(len(yte)), "test_churn_rate": round(float(yte.mean()), 4),
           "majority_baseline_accuracy": round(float(1 - yte.mean()), 4)}
    (RESULTS / "task4_comparison.json").write_text(json.dumps(out, indent=2))
    print(df.round(4).drop(columns="model").to_string()); print("BEST:", best, "thr", thr, meta["test_metrics_at_threshold"])


if __name__ == "__main__":
    main()
```